

Scheduling Irregular Tasks on Heterogeneous Platforms

LOOM: Interweaving Simulation and Visualization

João Barbosa and André Pereira (some contributions)

Table of Contents

1. Introduction & Foundations

1.1 Overview & Background

1.2 Directed Acyclic Graphs (DAGs)

1.3 Task Execution Time Variability - Irregular Workloads

1.4 Load Balancing Strategies

2. LOOM: Diceable Tasks Approach

2.1 Implementation

2.2 Applications

Introduction & Foundations

Overview & Background

Introduction & Overview

- **Context:** Modern HPC jobs are represented as directed acyclic graphs (DAGs) of tasks with precedence constraints.
- **Goal:** Efficiently schedule irregular tasks on heterogeneous resources.
- **Agenda:**
 - ▶ DAG & task scheduling fundamentals
 - ▶ Regular vs. irregular workloads
 - ▶ Workload estimation and irregularity (ϵ)
 - ▶ Load balancing techniques
 - ▶ LOOM: Diceable tasks approach
 - ▶ Case studies and results

High Performance Computing Challenges

- Modern scientific applications are increasingly complex
 - ▶ Numerical simulation is cornerstone for discovery
 - ▶ Increasing demands for computational resources
 - ▶ Massive datasets for analysis
- Scientific visualization is critical
 - ▶ Traditionally performed *post-hoc*
 - ▶ Shifting to *in situ* approaches
 - ▶ Tightly coupled with numeric simulation
- Resource allocation challenges
 - ▶ Under-subscription wastes resources
 - ▶ Over-subscription delays simulation

Directed Acyclic Graphs (DAGs)

What is a DAG?

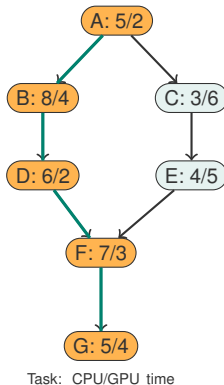
Definition (Directed Acyclic Graph (DAG))

A directed graph $G = (V, E)$ where:

- V is a set of nodes (tasks)
 - $E \subseteq V \times V$ is a set of directed edges (dependencies)
 - No directed cycles exist in the graph
-
- Each node represents a computational task
 - Each edge (u, v) means task v depends on u
 - DAGs encode precedence constraints for scheduling
 - DAGs can represent applications at different levels of granularity
 - ▶ Each task can be a DAG on its own
 - ▶ Simple patterns like fork-join are common

DAG Example

- Tasks as vertices, dependencies as edges
 - ▶ Task *C* cannot start until *A* finishes
 - ▶ Tasks *B* and *C* both depend only on *A*, so they can start as soon as *A* finishes and can run in parallel with each other
- **Critical path:** The longest dependency chain determines the minimum possible makespan
- Parallelism emerges from the wavefront of ready tasks



Critical path: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$

Critical Path in DAG Scheduling

Definition (Critical Path)

Given a DAG $G = (V, E)$ with task durations $t(v)$ for each $v \in V$, the *critical path length* L^* is defined as:

$$L^* = \max_{P \in \mathcal{P}} \sum_{v \in P} t(v)$$

where $\mathcal{P}$ is the set of all directed paths from an entry node (no predecessors) to an exit node (no successors) in G .

- L^* is the minimum possible makespan, even with infinite resources.
- Any valid schedule must have total duration at least L^* .
- Tasks on the critical path must be executed sequentially due to dependencies.
- Scheduling objective: minimize total execution time while respecting dependencies

The Scheduling Problem

- **Given:**

- ▶ DAG $G = (V, E)$ of tasks with precedence constraints
- ▶ Set of identical processors P
- ▶ Task execution time $t(v)$ for each task v

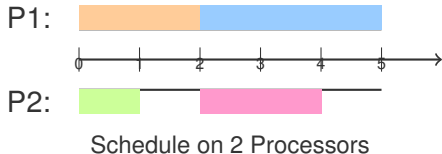
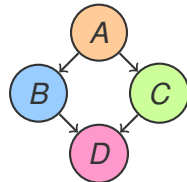
- **Find:**

- ▶ Processor assignment: which task runs where
- ▶ Start time for each task

- **Constraints:**

- ▶ Respect all precedence constraints in DAG
- ▶ One task per processor at any time
- ▶ Minimize total completion time (makespan)

Precedence DAG



Mathematical Formulation

Minimize $C_{\max}$

Subject to:

$$s_j \geq s_i + t_i \quad \forall (i, j) \in E$$

$$s_j \geq s_i + t_i \vee s_i \geq s_j + t_j \quad \text{if on same proc.}$$

$$\sum_{k=1}^m x_{i,k} = 1 \quad \forall i \in V$$

$$C_{\max} \geq s_i + t_i \quad \forall i \in V$$

Computational Complexity

- **Scheduling with precedence constraints:**
 - ▶ Even with unlimited processors: $O(n)$ solution exists
 - ▶ With limited processors: NP-hard
 - ▶ Optimal solutions computationally expensive
- **Key complexity results:**
 - ▶ Scheduling independent tasks on identical processors: NP-hard for minimizing makespan
 - ▶ Special case: For exactly 2 identical processors, LPT produces optimal schedules
 - ▶ For $m = 2$ processors: LPT algorithm gives $\frac{4}{3}$ -approximation ratio
 - ▶ Sort tasks by decreasing execution time
 - ▶ Assign each task to least loaded processor
 - ▶ Scheduling with precedence constraints on two processors: NP-complete

Approximation Algorithms for Scheduling

- **What is a $\frac{4}{3}$ -approximation?**

- ▶ For a minimization problem, an α -approximation algorithm ($\alpha > 1$) produces solutions with cost at most α times the optimal solution
- ▶ For LPT: $\text{makespan}_{\text{LPT}} \leq \frac{4}{3} \cdot \text{makespan}_{\text{optimal}}$
- ▶ Provides a guaranteed quality bound while running in polynomial time
- ▶ Example: If optimal schedule takes 30 time units, LPT guarantees a schedule of at most 40 time units

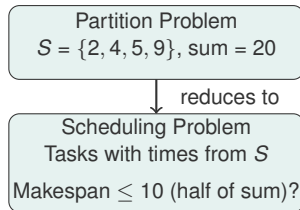
- **Why approximations matter:**

- ▶ Finding optimal schedules is often computationally prohibitive
- ▶ Approximation algorithms provide a balance between quality and efficiency
- ▶ The $\frac{4}{3}$ ratio is tight for LPT (worst-case instances exist that achieve this bound)

NP-Hardness: The Partition Problem

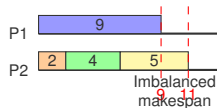
- **Partition Problem:**

- ▶ Given a set of positive integers
 $S = \{a_1, a_2, \dots, a_n\}$
- ▶ Can we partition S into two subsets with equal sum?
- ▶ Example: $S = \{2, 4, 5, 9\}$ with sum = 20
- ▶ No perfect partition exists: Closest is $\{9\} = 9$ and $\{2, 4, 5\} = 11$



- **Reduction to Scheduling:**

- ▶ Create n tasks with execution times
 $a_1, a_2, \dots, a_n$
- ▶ Schedule on 2 identical processors
- ▶ Question: Is makespan $\leq \frac{1}{2} \sum_{i=1}^n a_i$?
- ▶ This is equivalent to finding a perfect partition



Practical Implications of NP-Hardness

- **Why NP-hardness matters:**
 - ▶ Optimal scheduling is computationally prohibitive for real applications
 - ▶ Must trade optimality for efficiency in practice
 - ▶ Even simple cases (2 processors, independent tasks) are hard
- **Practical scheduling approaches:**
 - ▶ **List scheduling:** Prioritize tasks by some heuristic (e.g., LPT)
 - ▶ **Critical path scheduling:** Prioritize tasks on the critical path
 - ▶ **Work stealing:** Idle processors "steal" tasks from busy ones
 - ▶ **Work donation:** Processors donate tasks to help finish its tasks
- **Real-world examples:**
 - ▶ Intel TBB uses work stealing for load balancing
 - ▶ OpenMP runtime uses work sharing and stealing
 - ▶ Cilk uses provably efficient work-stealing scheduler

Task Execution Time Variability - Irregular Workloads

Work Item Execution Time Model

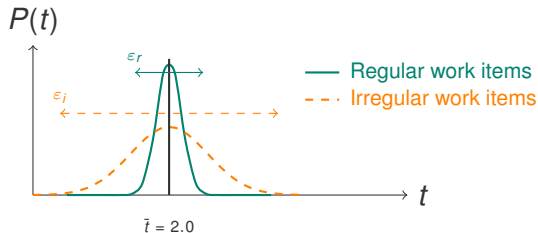
Definition (Work Item Execution Time)

The execution time of a work item w on resource r is modeled as:

$$t(w, r) = \frac{W_w}{p_r} \times (1 + \varepsilon_{w,r})$$

- W_w : Amount of work for work item w (e.g., operations, iterations)
- p_r : Processing capability of resource r (e.g., operations/sec)
- $\varepsilon_{w,r}$: Irregularity factor (captures execution time variance)
 - ▶ For regular work items: $\varepsilon_{w,r}$ is small (predictable execution)
 - ▶ For irregular work items: $\varepsilon_{w,r}$ is large (high execution variance)

Work Item Duration Distributions



- Work item execution time formula: $t(w, r) = \frac{W_w}{p_r} \times (1 + \varepsilon)$
- Where $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ with:
 - ▶ Regular work items: σ^2 is small $\Rightarrow$ predictable execution time
 - ▶ Irregular work items: σ^2 is large $\Rightarrow$ unpredictable execution time

Sources of Irregularity in HPC Applications

- **Data-dependent computation**
 - ▶ Adaptive mesh refinement
 - ▶ Sparse matrix operations
 - ▶ Particle simulations with varying density
 - ▶ Ray tracing with varying scene complexity
- **System-level variability**
 - ▶ Shared resource contention
 - ▶ OS jitter and interference
 - ▶ Power/thermal throttling
- **Heterogeneous hardware**
 - ▶ Different processing capabilities (CPU vs. GPU)
 - ▶ Communication bottlenecks

Load Balancing Strategies

Traditional Load Balancing Approaches

- **Static load balancing**

- ▶ Distribute work evenly at the beginning
- ▶ Works well for regular workloads
- ▶ Poor performance for irregular workloads

- **Dynamic load balancing**

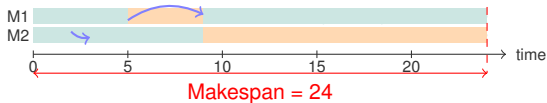
- ▶ Work stealing: idle processors take work from busy ones
- ▶ Work sharing: busy processors offload work to idle ones
- ▶ Adaptive partitioning: adjust work distribution during execution

- **Limitations**

- ▶ Overhead of work redistribution
- ▶ Task granularity constraints
- ▶ Difficulty in predicting execution time

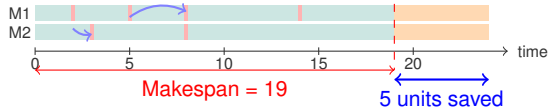
Scheduling Challenges for Irregular Tasks

Static Scheduling (Predetermined)



- Tasks assigned to processors before execution
- Long idle periods due to fixed assignments
- Task E's irregular execution time creates imbalance
- No adaptation to runtime conditions

Dynamic Scheduling (Runtime)



- Tasks assigned at runtime when dependencies satisfied
- Minimal scheduling overhead (red lines)
- Ready tasks (D, G, F) fill potential idle periods
- Large task E can be split between processors

Key Challenges with Irregular Tasks

1. Even with dynamic scheduling, some idle time remains due to DAG dependencies
2. Task granularity limits how much large tasks can be divided
3. Irregular execution times make static scheduling inefficient

LOOM: Diceable Tasks Approach

Relaxing the Task Granularity Constraint

- **Traditional approach:** Tasks are atomic units
 - ▶ A task must be executed in its entirety on a single resource
 - ▶ Once started, a task cannot be interrupted
- **LOOM approach:** Tasks can be "diced" into smaller units
 - ▶ A task can be partitioned into smaller sub-tasks at runtime
 - ▶ Sub-tasks can be scheduled independently
 - ▶ Allows filling small gaps in the schedule
- **Benefits:**
 - ▶ Better utilization of resources
 - ▶ Reduced idle time
 - ▶ More flexibility in handling irregular workloads

Dicing Mechanism in LOOM

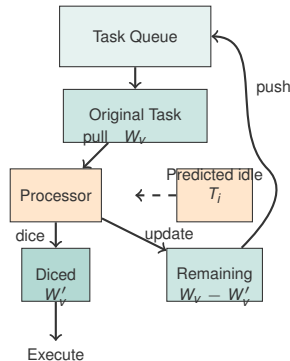
- **Workload partitioning:**

$$W(\tau'_v) = \frac{T_i * W(\tau_v)}{T_v} \quad (1)$$

- ▶ $W(\tau_v)$: Original task workload
- ▶ $W(\tau'_v)$: Partitioned task workload
- ▶ T_i : Predicted idle time
- ▶ T_v : Predicted task execution time

- **Workload prediction:**

- ▶ Uses a simple moving average model
- ▶ Collects performance data during execution
- ▶ Adapts to changing workload patterns



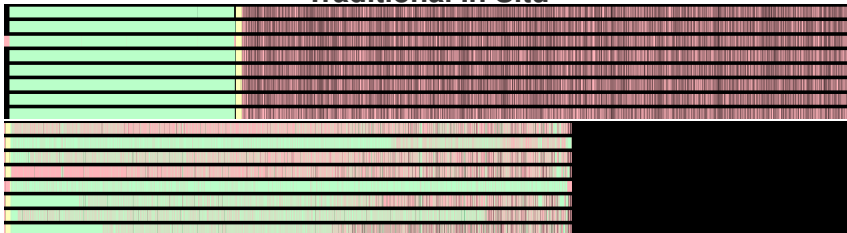
Implementation

LOOM Scheduler Implementation

- **Task hierarchy:**
 - ▶ Generic Task: Basic unit of computation
 - ▶ Diceable Task: Task that can be partitioned
- **Queue system:**
 - ▶ Global Not Ready Queue (GNRQ): Tasks with dependencies
 - ▶ Global Ready Queue (GRQ): Tasks ready for execution
 - ▶ Local Ready Queue (LRQ): Per-core scheduling queue
- **Scheduling policy:**
 - ▶ High priority tasks: Simulation tasks
 - ▶ Low priority tasks: Visualization tasks
 - ▶ Visualization tasks are scheduled when simulation threads are idle
- **Work stealing:**
 - ▶ Load balancing among cores
 - ▶ Reduces access contention to GRQ

LOOM Interweaving Strategy

Traditional In Situ



LOOM Approach

- Visualization tasks for time step $t - 1$ are interleaved with simulation tasks for time step t
- Idle time is utilized to perform visualization
- Overall execution time is reduced

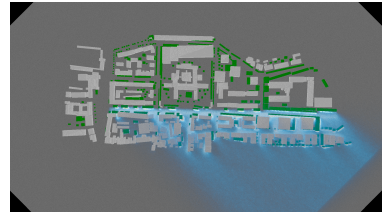
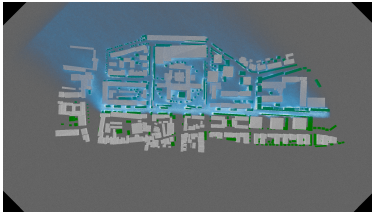
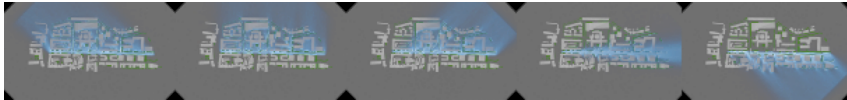
Applications

Test Application: VADIS

VADIS: Computational Fluid Dynamics

- Pollutant dispersion in atmosphere under variable wind conditions
- Two modules working together:
 - ▶ **FLOW:** Calculates wind velocity components using Navier-Stokes equations
 - ▶ **DISPER:** Computes 3D concentration field using Langrangian approach
- High irregularity due to complex urban morphologies
- Computational intensity varies across the domain and over time

VADIS Workload Distribution



Ideal Test Case for LOOM

Varying computational intensity creates natural idle periods that can be utilized for visualization tasks.

VADIS Performance Results

Grid size	T_s	T_{s+v}	$T_{s v}$	TTV Improv
3m	187.29	487.61	402.2	1.39x
1.5m	636.02	1097.94	929.89	1.57x
1m	1723.18	2382.19	1953.83	2.85x

Table: VADIS execution times (seconds)

- T_s : Simulation time only
- T_{s+v} : Sequential execution (simulation + visualization)
- $T_{s||v}$: LOOM interweaving approach
- TTV: Time to visualization
- TTV Improv: Improvement ratio of time to visualization ($\frac{T_{s+v} - T_s}{T_{s||v} - T_s}$)

Performance Analysis

Key Finding

Performance improvement increases with grid resolution
Up to **2.85x** faster visualization delivery

Impact of Grid Resolution

As computational intensity increases with finer grid resolution, LOOM's ability to interleave visualization tasks during idle periods becomes more effective, resulting in greater performance improvements.

- Finer grids (1m) show the greatest benefit from LOOM's approach
- More complex computations create more opportunities for task interleaving

Conclusions

- **LOOM:** A novel approach for irregular task scheduling
 - ▶ Relaxes the task atomicity constraint through dicing
 - ▶ Enables better resource utilization
 - ▶ Prioritizes simulation while fitting visualization in idle slots
- **Benefits:**
 - ▶ Up to 2.85x faster time to visualization
 - ▶ Minimizes or hides visualization workload cost
 - ▶ Scales well with increased workload irregularity
- **Future work:**
 - ▶ Improved idle time prediction models
 - ▶ Automatic detection of diceable tasks
 - ▶ Support for more complex heterogeneous architectures